

Infografía

Módulo 7 — Godot 4 • Sesión 1

GDScript + nodos + escenas + señales

UPB • Gestión I/2026

Plan de la clase

1. **¿Qué es Godot?** — un motor de juegos en una hora.
2. **El editor** — los cuatro paneles que importan.
3. **Nodos y escenas** — la única abstracción que Godot tiene.
4. **GDScript** — Python con `extends` y `@export`.
5. **Señales** — cómo dos nodos hablan sin conocerse.
6. **Las cuatro demos del proyecto** — `main.tscn` y los tres demos sueltos.

Hasta ahora todo fue Python: `arcade`, `pymunk`, transformaciones a mano. Hoy cambiamos de stack completo: editor visual, lenguaje propio, motor con renderer/física/audio integrado.

1. ¿Qué es Godot?

el motor

En una frase

“ Un **motor de juegos 2D/3D**, libre, multiplataforma, ~70 MB, con editor visual y lenguaje propio (GDScript). ”

- **Libre** — MIT, sin regalías, sin licencias.
- **Multiplataforma** — corre y exporta a Windows / macOS / Linux / Web / Android / iOS / consolas.
- **Liviano** — un solo binario, sin instalar.
- **Editor + runtime en la misma app** — F5 corre el proyecto.

No es "como Unity pero gratis". Tiene un modelo distinto: todo es un árbol de nodos.

¿Qué tiene adentro?

Listo para usarse

- Renderer 2D y 3D (Vulkan, GL Compat)
- Motor de física 2D y 3D
- Sistema de animación
- UI (Control nodes)
- Audio
- Networking
- Editor de tilemaps
- Editor de shaders

Lo que tienes que poner tú

- El comportamiento (scripts)
- Los assets (sprites, sonidos, modelos)
- El diseño del juego

*Lo que en los módulos 3-6 estábamos armando a mano (gravedad, colisiones, transformaciones) acá viene resuelto. Lo que tenemos que aprender es **cómo hablar con el motor.***

Juegos hechos con Godot

- **Brotato, Dome Keeper, Cassette Beasts** — indies completos.
- **Sonic Colors: Ultimate** — partes de la UI usan Godot.
- **Halls of Torment, Buckshot Roulette** — recientes.
- Cientos de jam games, tutoriales y proyectos pequeños.

Tier "no es solo un juguete" — está en producción.

El stack mental



De abajo hacia arriba: el motor carga un proyecto, que tiene escenas, que son árboles de nodos, opcionalmente con scripts pegados.

2. El editor

los cuatro paneles

La primera vez que abres Godot

Cuatro paneles importan:

1. **FileSystem** (abajo-izquierda) — los archivos del proyecto. `.tscn`, `.gd`, assets.
2. **Scene** (arriba-izquierda) — el árbol de nodos de la escena abierta.
3. **Inspector** (derecha) — las propiedades del nodo seleccionado.
4. **Output / Debugger** (abajo) — `print()` aparece acá. También errores.

El **viewport** en el centro es donde ves la escena.

Esto es lo mínimo. Hay 10+ paneles más (Animation, AudioBus, Shader, etc.) — los ignoramos hoy.

Atajos que vas a usar siempre

Atajo	Qué hace
F5	Correr el proyecto (la escena principal)
F6	Correr la escena actual (la que tienes abierta)
F8	Detener
Ctrl+S	Guardar escena
Ctrl+Shift+S	Guardar como
doble-click en <code>.tscn</code>	Abrir esa escena en el editor
doble-click en <code>.gd</code>	Abrir ese script en el editor

F6 es clave hoy — tres de las cuatro demos no están en main.tscn, las corremos solas.

3. Nodos y escenas

la única abstracción

¿Qué es un nodo?

Una pieza con:

- un **tipo** (`Sprite2D`, `Area2D`, `Timer`, `Label` ...),
- un **nombre** (lo que ves en la Scene dock),
- **propiedades** (visibles en el Inspector),
- cero o más **hijos** (también nodos).

```
Player (CharacterBody2D)    ← este nodo es de tipo CharacterBody2D
├─ Sprite2D                ← hijo
└─ CollisionShape2D        ← hijo
```

Hay ~200 tipos de nodos built-in. No hay que aprenderlos todos: empiezas con los que necesitas y vas sumando.

Nodos que vamos a usar hoy

Tipo	Para qué
Node2D	nodo 2D genérico — sirve como contenedor / raíz
Sprite2D	dibujar una textura
CharacterBody2D	cuerpo que tú controlas (player)
Area2D	detecta entradas/salidas, no choca físicamente
CollisionShape2D	la forma usada para colisiones
Timer	dispara una señal cada X segundos
Control, Label, Button	UI 2D

¿Qué es una escena?

Un árbol de nodos guardado en un archivo `.tscn`.

Una escena puede:

- ser la escena principal del juego (la que corre con F5),
- ser instanciada como hijo de otra escena (un coin dentro de main),
- correrse sola con F6.

Escena = unidad reutilizable. Todo lo que vas a querer aparecer varias veces (player, coin, enemy, bullet) lo armas como escena.

El árbol de `main.tscn`

main.tscn — lo que ve la Scene dock cuando lo abris

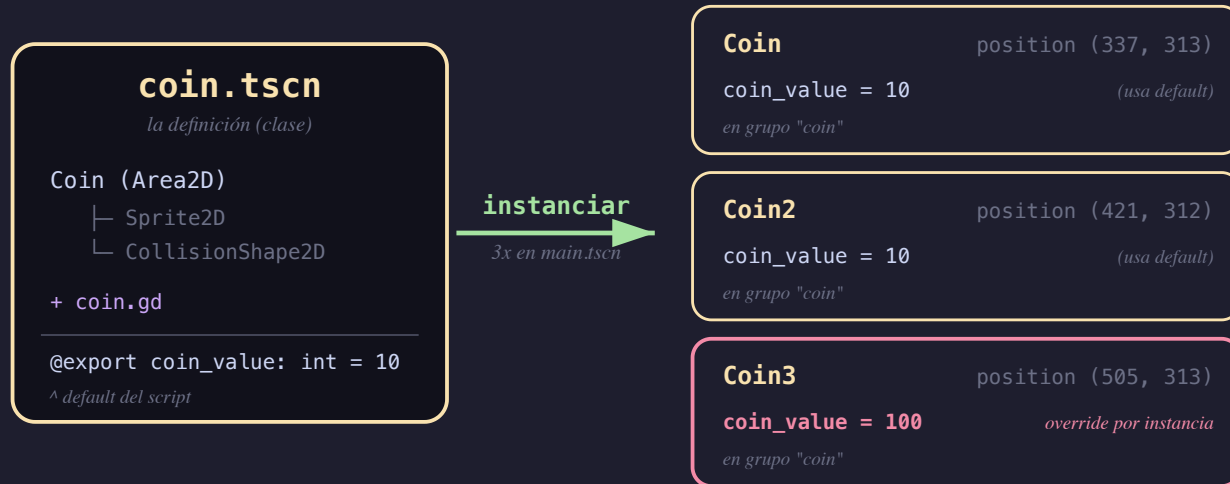


+ script = nodo con script .gd asociado

Player, Enemy y Coin son instancias de sus respectivas escenas. Control es una rama local — solo existe en main.

Escena = clase. Instancia = ejemplar.

una escena es la clase · cada instancia tiene su propio estado



el script define el default; la escena (o cada instancia) lo puede sobrescribir

Cambias `coin\_value` en el script → afecta a todas las coins.

Cambias `coin\_value` en una instancia → solo afecta esa.

Cómo se instancia (en el editor)

1. En la Scene dock, click derecho en el padre.
2. **"Instance Child Scene..."** → elegir el `.tscn`.
3. Ya aparece como hijo, configurable como cualquier otro nodo.

O en código (lo veremos en sesiones futuras):

```
var coin_scene: PackedScene = preload("res://scenes/coin.tscn")
var nueva_coin = coin_scene.instantiate()
add_child(nueva_coin)
```

4. GDScript

el lenguaje

GDScript en 30 segundos

- Sintaxis **muy parecida a Python** (indentación, sin `;`, sin `{}`).
- **Tipado opcional** — puedes escribir `var x = 10` o `var x: int = 10`.
- Diseñado para Godot: integra anotaciones (`@export`, `@onready`), señales y nodos como ciudadanos de primera clase.
- Archivo `.gd` = una clase. Se "pega" a un nodo en el editor.

```
extends CharacterBody2D      # de qué hereda este script

class_name Player           # opcional: le da un nombre global

@export var speed: int = 300

func _ready() -> void:
    print("hola, soy el player")
```

`extends` **y** `class_name`

```
class_name Player  
extends CharacterBody2D
```

- `extends X` — este script **es** un X (hereda de X). Obligatorio.
- `class_name Y` — opcional. Le pone un nombre global, para que otros scripts puedan hacer `if body is Y`.

Sin `class_name` el script funciona igual; solo no puedes hacer `is Player` en otros archivos.

Si dos scripts usan el mismo `class_name`, Godot tira error de compilación. Cada nombre es único en el proyecto.

@export — exponer una variable al Inspector

```
@export var speed: int = 300  
@export var health: int = 100  
@export_range(0, 500, 10) var jump_force: int = 250
```

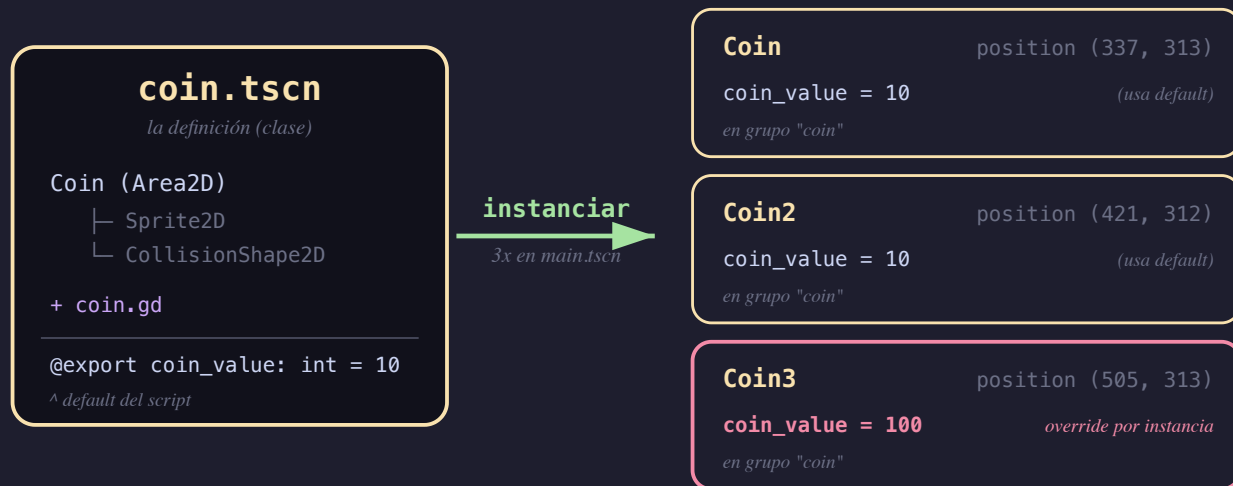
Lo que pasa:

- En el Inspector aparece un campo **Speed** editable.
- El valor que se guarda es **por instancia** (en el **.tscn**).
- En tiempo de ejecución, el script lee el valor de la escena, no el default del código.

@export es cómo el editor "ve" tu código.

Override por instancia

una escena es la clase · cada instancia tiene su propio estado

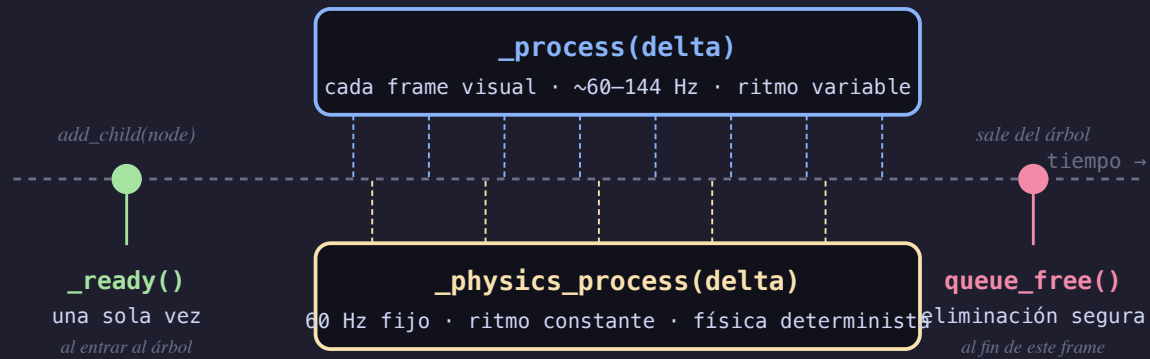


el script define el default; la escena (o cada instancia) lo puede sobrescribir

El script dice `coin_value = 10`. La escena `coin.tscn` también dice 10 (hereda). Pero en `main.tscn` puedes tener una `Coin3` con `coin_value = 100` sin tocar el script. **El script define, la escena tunea.**

Lifecycle: qué se llama y cuándo

cuándo se llama cada callback en la vida de un nodo



`_process` para lógica visual · `_physics_process` para movimiento + colisiones

regla práctica: si llama a `move_and_slide()` o toca física, va en `_physics_process`

`_process` vs `_physics_process`

```
func _process(delta: float) -> void:
    # cada frame visual (60-144 Hz, variable)
    rotation += delta * 0.5

func _physics_process(delta: float) -> void:
    # cada tick de física (60 Hz, fijo)
    velocity.x = direction * speed
    move_and_slide()
```

- `_process` para lo **visual** (rotaciones decorativas, parpadeo, contadores).
- `_physics_process` para **movimiento y colisiones** (todo lo que use `move_and_slide`, `velocity`, etc.).

Si pones `move_and_slide()` dentro de `_process`, el juego va más lento o más rápido según los FPS. Por eso física = ritmo fijo.

Input.get_axis — teclado en una línea

```
func _physics_process(delta: float) -> void:
    var direction = Input.get_axis("ui_left", "ui_right")
    # direction = -1 si solo ui_left
    # direction = 0 si nada o ambos
    # direction = +1 si solo ui_right

    velocity.x = direction * speed
    move_and_slide()
```

Las acciones `ui_left`, `ui_right`, `ui_up`, `ui_down`, `ui_accept` vienen con Godot.

*Para agregar nuevas (ej. `"jump"`, `"fire"`): **Project** → **Project Settings** → **Input Map**.*

5. Señales

cómo dos nodos hablan sin conocerse

El problema

```
Player ←—— ¿cómo se entera la UI cuando el player agarra una coin?  
Coin   ———→ ¿podemos hacer que la Coin tenga una referencia al Score Label?
```

Opción mala: que la `Coin` tenga un `@export var ui_label: Label`. Ahora todas las coins necesitan saber dónde está la UI. Si renombras el Label, se rompe todo.

Opción Godot: señales. La Coin **avisa** "fui agarrada, valor 10" — sin saber quién escucha. La UI **se suscribe** al aviso.

Patrón Observer: el emisor no sabe quién escucha. Acoplamiento mínimo.

Definir, emitir, escuchar

```
# 1. definir
signal coin_collected(value: int)

# 2. emitir (en algún lugar del código del emisor)
coin_collected.emit(coin_value)

# 3. escuchar (desde el listener)
coin.coin_collected.connect(on_coin_collected)

func on_coin_collected(value: int) -> void:
    current_score += value
```

La señal es como un evento. `emit()` dispara. `connect()` suscribe. Múltiples listeners pueden conectarse a la misma señal.

Cómo fluye en `main.tscn`

señales = comunicación sin acoplamiento · el emisor no sabe quién escucha



estilo 1 · conexión en editor

click derecho en Signals → Connect

```
en coin.tscn:  
[connection signal="body_entered"  
from="." to="." method="_on_body_entered"]
```

estilo 2 · conexión en código

.connect() desde un script

```
en control.gd::_ready():  
coin.coin_collected.connect(  
on_coin_collected)
```

Dos formas de conectar

En el editor

1. Selecciona el nodo emisor.
2. Panel **Node** → **Signals** (derecha).
3. Doble-click en la señal.
4. Elegir nodo destino + método.

Queda guardado dentro del `.tscn`:

```
[connection signal="body_entered"  
  from="." to="."  
  method="_on_body_entered"]
```

En código

```
func _ready() -> void:  
    $Timer.timeout.connect(explode)
```

O recorriendo nodos:

```
for coin in get_tree() \  
    .get_nodes_in_group("coin"):  
    coin.coin_collected.connect(  
        on_coin_collected)
```

Las dos sirven. Editor = más visible. Código = más explícito en diff. No hay una regla universal.

Señales built-in que vamos a ver hoy

Señal	Quién la emite	Cuándo
<code>body_entered(body)</code>	<code>Area2D</code>	un body físico entra al área
<code>body_exited(body)</code>	<code>Area2D</code>	un body físico sale
<code>timeout</code>	<code>Timer</code>	el timer terminó
<code>pressed</code>	<code>Button</code>	se hizo click

```
# enemy.tscn conecta body_entered → _on_body_entered (en editor)
func _on_body_entered(body: Node2D) -> void:
    if body.has_method("take_damage"):
        body.take_damage(damage_amount)
```

Grupos — etiquetas para nodos

```
# en main.tscn cada coin pertenece al grupo "coin"
# en player.gd::_ready: add_to_group("player")

# encontrar todos:
var coins = get_tree().get_nodes_in_group("coin")

# preguntar si pertenece:
if body.is_in_group("player"):
    pass
```

Útil para "encuentro todos los enemigos", "soy un proyectil", etc. — sin tener que mantener arrays manualmente.

@onready y \$nodeName

```
@onready var score_label: Label = $Score
```

- `$Score` es atajo para `get_node("Score")` — busca un hijo con ese nombre.
- `@onready` **demora** la asignación hasta que el nodo esté en el árbol.
- Sin `@onready`, `var score_label = $Score` intentaría buscar antes de tiempo y daría `null`.

Regla práctica: si referencias un hijo, usa `@onready`.

`queue_free()` — eliminar un nodo

```
func _on_body_entered(body: Node2D) -> void:
    if body is Player:
        coin_collected.emit(coin_value)
        queue_free()      # me elimino al fin de este frame
```

`queue_free()` espera al final del frame. `free()` es inmediato — peligroso si alguien más está iterando sobre ese nodo.

Regla: siempre `queue_free()` salvo que sepas exactamente lo que haces.

6. Las cuatro demos

del proyecto `01_gdscript_basics`

`main.tscn` — F5

El "juego" del día. Mario + 3 coins + 1 enemy + UI.

F5 → corre la escena principal del proyecto

Conceptos que aparecen, todos a la vez:

- Instancias (3x Coin del mismo .tscn).
- Override por instancia (Player.speed = 800 vs script 300).
- `_physics_process` + `Input.get_axis` + `move_and_slide`.
- Señal built-in `body_entered` (conectada en editor).
- Señal custom `coin_collected` (conectada en código desde control.gd).
- Grupos (`"coin"`, `"player"`).
- `take_damage` con duck typing (`body.has_method(...)`).

bullet.tscn — F6

F6 → corre solo esta escena

Una **fireball** que se mueve a velocidad constante. Solo 8 líneas:

```
extends Area2D

const SPEED = 600.0

func _process(delta: float) -> void:
    position.x += SPEED * delta
```

El ejemplo más chico posible de `_process`. Sin física, sin señales, sin nada.

timer_example.tscn — F6

Bomba con un `Timer` adentro:

```
extends Node2D

func _ready() -> void:
    $Timer.timeout.connect(explode)
    $Timer.start()
    print("Bomb planted")

func explode():
    print("BOOM! 💣")
    queue_free()
```

El nodo Timer es un subnodo del Node2D. Su propiedad `wait_time = 5.0` está configurada en el editor (no hace falta hacerlo en código).

control.tscn — F6

UI mínima: un `Button` + un `Label`. Click → label cambia.

```
extends Label

func _on_button_pressed() -> void:
    text = "hola bola!"
```

La conexión `Button.pressed → Label._on_button_pressed` está hecha **en el editor**.

```
[connection signal="pressed"
 from="Button" to="Label"
 method="_on_button_pressed"]
```

Cuatro líneas de GDScript + una conexión en editor = botón funcional.

7. Ejercicios en clase

10 minutos antes de cerrar

Prueba esto

1. Abre `main.tscn` y selecciona `Player`. En el Inspector, cambia `Speed` a 200. Corre con F5. ¿Notas la diferencia?
2. Selecciona `Coin3` en `main.tscn`. En el Inspector, cambia `Coin Value` a 100. Corre y agarra las tres coins en orden — ¿con qué score te quedas?
3. Abre `timer_example.tscn`. Selecciona el nodo `Timer`. Cambia `Wait Time` a 1.5. F6 y mira el Output.
4. **Más difícil:** en `coin.gd`, cambia `coin_value` default a 50 y remueve el override en `Coin3`. ¿Qué pasa con el score?

Lo que viene en sesión 2

- **2D nodes + input + movement** — completamos el player: gravedad, salto, cámara que sigue, animaciones de sprite.
- Vamos a separar el **control de input** del **cuerpo del personaje** (patrón sacado de `bunny/player_control.tscn`).
- Mini-nivel con plataformas para caminar y saltar.

Sesión 1: viste los pedazos. Sesión 2: los conectamos en algo que se siente como un juego.

Resumen

los puntos que tienes que llevarte hoy

En una página

1. **Godot = motor + editor en una sola app.** F5 corre el proyecto.
2. **Todo es un nodo.** Tipos built-in (`Sprite2D`, `Area2D`, `Timer`, ...) + tus scripts.
3. **Escena = árbol de nodos en un `.tscn`.** Instanciable y reutilizable.
4. **GDScript = Python con `extends` + `@export`.** El script define los defaults, la escena los puede sobrescribir.
5. `_ready` una vez, `_process` cada frame visual, `_physics_process` 60 Hz fijo.
6. **Señales = comunicación sin acoplamiento.** Conectables en editor o en código.
7. **Grupos = etiquetas** para encontrar/clasificar nodos sin referencias hardcodeadas.

Si entendiste esos siete puntos, la próxima sesión es construcción, no novedad conceptual.